

# Reviewer Pack — 2-Sylow Rank of Critical Group Bounds Tseitin DPLL Size

Entry eb9e15c9c9db · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-17 08:50:59 UTC

## 2-Sylow Rank of Critical Group Bounds Tseitin DPLL Size

**Entry ID:** eb9e15c9c9db **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-17 08:50:59 UTC

### 1. Conjecture

**Field A** (mathematical branch): Algebraic combinatorics of the critical (sandpile) group  $K(G)$ : specifically the 2-Sylow rank  $r_2(G) = \dim_{\mathbb{F}_2} \ker(\tilde{L}_G \bmod 2)$  of the reduced graph Laplacian, in the Björner-Lovász-Shor / Wood-Mészáros tradition on cokernels of random integer matrices; distinct from the well-studied invariant  $|K(G)| = \tau(G)$  and essentially absent from Tseitin proof-complexity literature **Field B** (complexity object): Tree-like Resolution / DPLL refutation size for Tseitin XOR formulas  $T(G, \omega)$  on 3-regular graphs (Urquhart-Ben-Sasson-Razborov regime), via the cycle-space-mod-2 mechanism underlying expander Tseitin lower bounds

#### Statement:

For every connected 3-regular graph  $G$  on  $n$  vertices and every charge  $\omega: V \rightarrow \mathbb{F}_2$  with  $\sum \omega \equiv 1 \pmod{2}$ , let  $\tilde{L}_G$  be the reduced graph Laplacian (Laplacian with one row/column removed) and define the 2-Sylow rank  $r_2(G) := (n-1) - \text{rank}_{\mathbb{F}_2}(\tilde{L}_G \bmod 2)$ . Let  $N_{\text{DPLL}}(T(G, \omega))$  be the number of nodes in the canonical DPLL refutation tree of the Tseitin CNF (with unit propagation, branch on lowest-index unassigned variable). Conjecture:  $\log_2 N_{\text{DPLL}}(T(G, \omega)) \geq (1/4) \cdot (n-1-r_2(G))$  for every such  $(G, \omega)$ ; equivalently, a single  $(G, \omega)$  with  $\log_2 N_{\text{DPLL}} < (1/4)(n-1-r_2(G))$  falsifies it.

#### Rationale (proposer's reasoning):

Tseitin refutation hardness is governed by the cycle-space mod 2 of  $G$ , and  $r_2(G)$  precisely captures how many independent  $\mathbb{F}_2$ -degenerate directions the Laplacian pairing has on that cycle space; thus low  $r_2(G)$  (full  $\mathbb{F}_2$ -rigidity) should force DPLL to traverse all cycle-space cosets while high  $r_2$  opens algebraic short-cuts. This invariant is graph-structural (not a truth-table predicate), bypassing natural proofs, and is computed by elementary  $\mathbb{F}_2$  linear algebra rather than ring extensions, so it does not algebrize. It connects an under-utilised corner of algebraic combinatorics (2-part of Jacobian cokernel distributions) to the established expander-Tseitin lower-bound machinery.

**Taxonomy category:** PROOF\_COMPLEXITY\_TSEITIN (status at proposal time: alive)

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** 187c7610c63b318a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

### Acceptance criterion:

Across 150 instances (30 random 3-regular graphs  $\times$  5 sizes  $n \in \{6, 8, 10, 12, 14\}$ , each with a uniform odd-weight charge  $\omega$ ), the inequality  $\log_2 N_{\text{DPLL}}(T(G, \omega)) \geq 0.25 \cdot (n-1-r_2(G))$  must hold for every instance; a single violation falsifies.

## 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|---------|------------|------------------|------------------|
| RELATIVIZATION | SAFE    | 0.95       | SAFE             | SAFE             |
| NATURAL_PROOFS | SAFE    | 0.95       | SAFE             | SAFE             |
| ALGEBRIZATION  | SAFE    | 0.86       | SAFE             | SAFE             |
| KARP_LIPTON    | SAFE    | 0.97       | SAFE             | SAFE             |

## 4. Novelty audit

**Verdict:** NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

**Search queries** (3): -Tseitin formulas resolution lower bounds cycle space F<sub>2</sub> Laplacian-critical group sandpile 2-Sylow rank Laplacian cokernel 3-regular graph-DPLL tree-like resolution Tseitin expander binary Laplacian rank

## 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only,  $\leq 90$ s wall time per cycle **Execution:** rc=124, elapsed=240.1s

### 5.1 Generated Python source

```
import sys
import random
import math
import itertools
from collections import defaultdict

def generate_3_regular_graph(n, seed):
    random.seed(seed)
    if n % 2 != 0:
        raise ValueError("n must be even for 3-regular graphs")
    edges = []
    vertices = list(range(n))
    degrees = [0] * n
    while len(edges) < 3 * n // 2:
        u, v = random.sample(vertices, 2)
        if degrees[u] < 3 and degrees[v] < 3 and u != v and (u, v) not in edges and (v, u) not in edges:
            edges.append((u, v))
            degrees[u] += 1
            degrees[v] += 1
    return edges

def generate_charge(n, seed):
    random.seed(seed)
    charge = [random.randint(0, 1) for _ in range(n)]
```

```

    if sum(charge) % 2 == 0:
        charge[0] = 1 - charge[0]
    return charge

def matrix_multiply(A, B):
    n = len(A)
    m = len(B[0])
    p = len(B)
    result = [[0 for _ in range(m)] for _ in range(n)]
    for i in range(n):
        for j in range(m):
            for k in range(p):
                result[i][j] ^= A[i][k] & B[k][j]
    return result

def gaussian_elimination(matrix):
    n = len(matrix)
    for i in range(n):
        if matrix[i][i] == 0:
            for j in range(i + 1, n):
                if matrix[j][i] == 1:
                    matrix[i], matrix[j] = matrix[j], matrix[i]
                    break
            if matrix[i][i] == 0:
                continue
            for j in range(i + 1, n):
                if matrix[j][i] == 1:
                    for k in range(i, n):
                        matrix[j][k] ^= matrix[i][k]
    rank = sum(1 for i in range(n) if any(matrix[i][j] for j in range(n)))
    return rank

def compute_r2(G, n):
    adjacency = [[0] * n for _ in range(n)]
    for u, v in G:
        adjacency[u][v] ^= 1
        adjacency[v][u] ^= 1
    L = [[0] * n for _ in range(n)]
    for i in range(n):
        L[i][i] = sum(adjacency[i])
        for j in range(n):
            L[i][j] ^= adjacency[i][j]
    L_tilde = [row[:n-1] for row in L[:n-1]]
    rank = gaussian_elimination(L_tilde)
    return (n - 1) - rank

def generate_tseitin_cnf(G, omega, n):
    clauses = []
    for u, v in G:
        clauses.append([u, v, n + len(clauses)])
    for i in range(n):
        if omega[i] == 1:
            clauses.append([i])
    return clauses

def dpll(clauses, assignment, node_count):

```

```

node_count[0] += 1
if not clauses:
    return True, node_count
for clause in clauses:
    if not clause:
        return False, node_count
unit_clauses = [c[0] for c in clauses if len(c) == 1]
for lit in unit_clauses:
    if not assignment.get(lit, None) is None and assignment[lit] != 1:
        return False, node_count
    assignment[lit] = 1
    new_clauses = []
    for clause in clauses:
        if lit not in clause:
            new_clauses.append([x for x in clause if x != -lit])
    satisfied, node_count = dpll(new_clauses, assignment.copy(), node_count)
    if satisfied:
        return True, node_count
    assignment[lit] = 0
    new_clauses = []
    for clause in clauses:
        if -lit not in clause:
            new_clauses.append([x for x in clause if x != lit])
    satisfied, node_count = dpll(new_clauses, assignment.copy(), node_count)
    if satisfied:
        return True, node_count
unassigned = [lit for lit in range(len(assignment)) if assignment.get(lit, None) is None]
if not unassigned:
    return False, node_count
lit = min(unassigned, key=lambda x: abs(x))
assignment[lit] = 1
new_clauses = []
for clause in clauses:
    if lit not in clause:
        new_clauses.append([x for x in clause if x != -lit])
satisfied, node_count = dpll(new_clauses, assignment.copy(), node_count)
if satisfied:
    return True, node_count
assignment[lit] = 0
new_clauses = []
for clause in clauses:
    if -lit not in clause:
        new_clauses.append([x for x in clause if x != lit])
satisfied, node_count = dpll(new_clauses, assignment.copy(), node_count)
return satisfied, node_count

def run_trial(seed):
    n_values = [6, 8, 10, 12, 14]
    metric_values = []
    instances_tested = 0
    conjecture_holds = True
    counterexample = ""
    for n in n_values:
        for _ in range(6):
            G = generate_3_regular_graph(n, seed)
            omega = generate_charge(n, seed)

```

```

r2 = compute_r2(G, n)
clauses = generate_tseitin_cnf(G, omega, n)
node_count = [0]
satisfied, node_count = dpll(clauses, {}, node_count)
if not satisfied:
    N_DPLL = node_count[0]
    metric_value = math.log2(N_DPLL) if N_DPLL > 0 else 0
    bound = 0.25 * (n - 1 - r2)
    if metric_value < bound:
        conjecture_holds = False
        counterexample = f"n={n}, r2={r2}, N_DPLL={N_DPLL}, log2(N_DPLL)={metric_value}"
        break
    metric_values.append(metric_value)
    instances_tested += 1
if not conjecture_holds:
    break
if not conjecture_holds:
    break
if not metric_values:
    metric_values = [0]
return {
    "metric_name": "log2(N_DPLL)",
    "metric_value": sum(metric_values) / len(metric_values) if metric_values else 0,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = sys.argv[1:] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41]
    metric_values = []
    instances_tested = 0
    conjecture_holds_all = True
    counterexample = ""
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        metric_values.append(result["metric_value"])
        instances_tested += result["instances_tested"]
        if not result["conjecture_holds"]:
            conjecture_holds_all = False
            counterexample = result["counterexample"]
            break
    if conjecture_holds_all:
        mean = sum(metric_values) / len(metric_values)
        std = math.sqrt(sum((x - mean) ** 2 for x in metric_values) / len(metric_values))
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction=1.0")
    elif counterexample:
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={seeds[0]}")
    else:
        print("RESULT: INCONCLUSIVE reason=budget_exceeded n_tested=150")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

TIMEOUT after 240s

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

The test timed out after 240s (returncode 124) without producing a RESULT line or any instance data, so the pre-registered support condition over 150 instances cannot be evaluated. | next: Re-run with a longer timeout and/or reduce the size range (e.g.,  $n \in \{6,8,10,12\}$ ) and cache DPLL counts to obtain complete data on all 150 instances.

## 11. Audit log (LLM calls)

**Total LLM calls:** 9

| # | Phase           | Provider   | Model            | Tokens in | out | Latency (ms) |
|---|-----------------|------------|------------------|-----------|-----|--------------|
| 1 | propose         | claude_max | opus             | 0         | 0   | 266983       |
| 2 | preregistration | claude_max | opus             | 0         | 0   | 5371         |
| 3 | novelty         | claude_max | opus             | 0         | 0   | 3336         |
| 4 | novelty         | claude_max | opus             | 0         | 0   | 6779         |
| 5 | test_gen        | mistral    | codestral-latest | 0         | 0   | 315095       |
| 6 | test_gen        | mistral    | codestral-latest | 0         | 0   | 312724       |
| 7 | test_gen        | mistral    | codestral-latest | 0         | 0   | 317403       |
| 8 | test_gen        | mistral    | codestral-latest | 0         | 0   | 314461       |
| 9 | judge           | claude_max | opus             | 0         | 0   | 4209         |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 1546362 ms total latency. Provider mix: {'claude\_max': 5, 'mistral': 4}

(full prompt+response transcripts available in *research/audit/eb9e15c9c9db.jsonl*)

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/eb9e15c9c9db.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** *research/pvsnp\_sandbox/test\_\*.py* (per-cycle)

**Replay tarball:** *research/replay/eb9e15c9c9db.tar.gz* (if generated)